

Name: J O Joanna Divine

Reg No: 192572003

Course Code: CSA0918

Evaluation Plan: Adopting Recent Collection Framework Changes

Question: A software team is upgrading to a modern JDK and wants to adopt newer collection features without breaking old modules. Prepare an evaluation plan covering API compatibility, mutability, iteration order, null behaviour and performance testing.

Background

Since JDK 9, the Collection Framework added convenience factory methods such as `List.of()`, `Set.of()` and `Map.of()` to quickly create fixed collections. These behave differently from the classic `ArrayList`/`HashSet`/`HashMap` that older modules were written against, so they cannot be swapped in blindly. Before rollout, each of the five areas below needs to be checked and the risk documented.

1. API Compatibility

- 1) Identify every place in the old modules that stores results in a `List/Set/Map` returned from a shared utility method.
- 2) Check whether the caller only reads the collection (safe) or later calls `add/remove/put` on it (risk).
- 3) Run the existing unit test suite unchanged against the new return type and record any failures.
- 4) Where a method's return type changes from `ArrayList` to `List.of()`, keep the method signature as the interface type (`List`, `Set`, `Map`) so callers are not affected at compile time.

2. Mutability

- 1) Assume factory-created collections (`List.of`, `Set.of`, `Map.of`, `List.copyOf`) are read-only.
- 2) Search the codebase for `.add()`, `.remove()`, `.put()`, `.set()` calls on any collection that may come from a factory method.
- 3) If a module genuinely needs to modify the collection later, wrap it: `new ArrayList<>(List.of(...))`.
- 4) Add a unit test that intentionally calls `add()` on the new collection and asserts that `UnsupportedOperationException` is thrown, so a future code change cannot silently reintroduce mutation.

3. Iteration Order

- 1) Do not rely on the order of `HashSet`, `HashMap` or `Set.of()/Map.of()` anywhere in business logic.
- 2) Where order matters (e.g. display order, report generation), switch explicitly to `LinkedHashSet`/`LinkedHashMap` or sort before use.
- 3) Note that `Set.of()/Map.of()` intentionally randomise iteration order between JVM runs (as a safeguard against code depending on it) — this is stricter than `HashSet`, which is merely unspecified.
- 4) Re-run order-sensitive tests a few times locally to catch any hidden order dependency before release.

4. Null Behaviour

- 1) Check every module for code that stores null as a placeholder value inside a `List`, `Set` or `Map`.

- 2) Factory methods (List.of, Set.of, Map.of) reject null elements/keys/values and throw NullPointerException immediately at creation time.
- 3) Replace null placeholders with Optional.empty() or a sentinel value before migrating that data to a factory-created collection.
- 4) Treat this as a fail-fast improvement: catching a bad null at creation time is safer than an old NullPointerException surfacing later during iteration.

5. Performance Testing

- 1) Benchmark collection creation time and iteration time for the old and new approaches using the same data size the module sees in production.
- 2) Use System.nanoTime() around isolated blocks, and repeat each measurement a few times to smooth out JIT warm-up noise.
- 3) Expect factory-created collections to build faster and use less memory (compact internal representation) but perform similarly on plain iteration.
- 4) Record the numbers so the change can be justified with evidence, not just assumption.

6. Supporting Java Code

The class below exercises all five checks above and prints the result of each so the findings can be verified directly.

```
import java.util.*;

public class CollectionEvaluationDemo {

    public static void main(String[] args) {

        System.out.println("=====");
        System.out.println(" 1. API COMPATIBILITY CHECK");
        System.out.println("=====");
        List<String> oldStyleList = new ArrayList<>();
        oldStyleList.add("Order1");
        oldStyleList.add("Order2");
        System.out.println("ArrayList (old style) works as before: " +
oldStyleList);

        List<String> newStyleList = List.of("Order1", "Order2");
        System.out.println("List.of() (new factory method) reads fine: " +
newStyleList);
        System.out.println("Conclusion: Read-only usage is compatible;
write usage is not (see test 2).");

        System.out.println();
        System.out.println("=====");
        System.out.println(" 2. MUTABILITY CHECK");
        System.out.println("=====");
        try {
            newStyleList.add("Order3");
            System.out.println("Add succeeded (unexpected)");
        } catch (UnsupportedOperationException e) {
```

```

        System.out.println("List.of() is IMMUTABLE -> add() threw: " +
e.getClass().getSimpleName());
    }
    oldStyleList.add("Order3");
    System.out.println("ArrayList is still MUTABLE -> add() succeeded:
" + oldStyleList);

    System.out.println();
    System.out.println("=====");
    System.out.println(" 3. ITERATION ORDER CHECK");
    System.out.println("=====");
    Set<String> hashSet = new HashSet<>(Arrays.asList("Delhi",
"Chennai", "Mumbai", "Pune"));
    Set<String> linkedHashSet = new
LinkedHashSet<>(Arrays.asList("Delhi", "Chennai", "Mumbai", "Pune"));
    Set<String> factorySet = Set.of("Delhi", "Chennai", "Mumbai",
"Pune");
    System.out.println("HashSet order      : " + hashSet);
    System.out.println("LinkedHashSet order  : " + linkedHashSet);
    System.out.println("Set.of() order      : " + factorySet);
    System.out.println("Conclusion: HashSet and Set.of() do NOT
guarantee insertion order,");
    System.out.println("                and Set.of() order can even change
between JVM runs.");

    System.out.println();
    System.out.println("=====");
    System.out.println(" 4. NULL BEHAVIOUR CHECK");
    System.out.println("=====");
    List<String> arrayListWithNull = new ArrayList<>();
    arrayListWithNull.add("A");
    arrayListWithNull.add(null);
    System.out.println("ArrayList accepts null: " +
arrayListWithNull);
    try {
        List<String> factoryListWithNull = List.of("A", null);
        System.out.println("List.of() accepted null (unexpected)");
    } catch (NullPointerException e) {
        System.out.println("List.of() REJECTS null -> threw: " +
e.getClass().getSimpleName());
    }

    System.out.println();
    System.out.println("=====");
    System.out.println(" 5. PERFORMANCE CHECK (1,000,000 elements)");
    System.out.println("=====");
    int n = 1_000_000;

    long t1 = System.nanoTime();
    List<Integer> arrayList = new ArrayList<>();
    for (int i = 0; i < n; i++) arrayList.add(i);
    long t2 = System.nanoTime();
    System.out.println("ArrayList build time      : " + (t2 - t1) /
1_000_000 + " ms");

```

```

        Integer[] data = arrayList.toArray(new Integer[0]);
        long t3 = System.nanoTime();
        List<Integer> immutableList = List.of(data);
        long t4 = System.nanoTime();
        System.out.println("List.of() build time      : " + (t4 - t3) /
1_000_000 + " ms");

        long t5 = System.nanoTime();
        long sum1 = 0;
        for (int x : arrayList) sum1 += x;
        long t6 = System.nanoTime();
        System.out.println("ArrayList iteration time  : " + (t6 - t5) /
1_000_000 + " ms");

        long t7 = System.nanoTime();
        long sum2 = 0;
        for (int x : immutableList) sum2 += x;
        long t8 = System.nanoTime();
        System.out.println("List.of() iteration time  : " + (t8 - t7) /
1_000_000 + " ms");
    }
}

```

7. Output

Command Prompt

```

C:\Users\Joanna\CollectionsDemo>javac CollectionEvaluationDemo.java
C:\Users\Joanna\CollectionsDemo>java CollectionEvaluationDemo
=====
1. API COMPATIBILITY CHECK
=====
ArrayList (old style) works as before: [Order1, Order2]
List.of() (new factory method) reads fine: [Order1, Order2]
Conclusion: Read-only usage is compatible; write usage is not (see test 2).

=====
2. MUTABILITY CHECK
=====
List.of() is IMMUTABLE -> add() threw: UnsupportedOperationException
ArrayList is still MUTABLE -> add() succeeded: [Order1, Order2, Order3]

=====
3. ITERATION ORDER CHECK
=====
HashSet order      : [Delhi, Chennai, Pune, Mumbai]
LinkedHashSet order : [Delhi, Chennai, Mumbai, Pune]

```

```
Command Prompt
Set.of() order      : [Delhi, Mumbai, Pune, Chennai]
Conclusion: HashSet and Set.of() do NOT guarantee insertion order,
              and Set.of() order can even change between JVM runs.

=====
4. NULL BEHAVIOUR CHECK
=====
ArrayList accepts null: [A, null]
List.of() REJECTS null -> threw: NullPointerException

=====
5. PERFORMANCE CHECK (1,000,000 elements)
=====
ArrayList build time      : 80 ms
List.of() build time      : 14 ms
ArrayList iteration time  : 34 ms
List.of() iteration time  : 34 ms
C:\Users\Joanna\CollectionsDemo>
```

8. Findings and Recommendation

- API compatibility: safe for read-only callers; any caller that mutates a returned collection will break and must be identified before migration.
- Mutability: confirmed — `List.of()` throws `UnsupportedOperationException` on `add()`, while `ArrayList` still allows it.
- Iteration order: confirmed unreliable — `HashSet`, `LinkedHashSet` and `Set.of()` each printed a different order for the same four elements.
- Null behaviour: confirmed — `ArrayList` silently accepted null, `List.of()` rejected it immediately with `NullPointerException`.
- Performance: `List.of()` built noticeably faster than `ArrayList` for 1,000,000 elements; iteration speed was about the same.

Recommendation: the new factory methods can be adopted for internal, read-only, fixed-size data (constants, lookup tables, method return values that callers only read). Existing mutable/nullable data structures and any order-dependent logic should be migrated only after the specific call sites are checked, since these are the areas most likely to break old modules.